WIRELESS MOBILE COMMUNICATION (WMC) – PRACTICAL JOURNAL

Packet Capture and Protocol Analysis using Wireshark (HTTP, TCP, UDP, DNS, ARP, ICMP)

Jatin Lachhani

Aim

To capture live network traffic on a Wi-Fi interface using Wireshark, generate traffic with ping and web browsing (HTTP and HTTPS sites), and then analyse individual HTTP, TCP, UDP, DNS, ARP and ICMP packets using display filters, the Packet List, Packet Details and Packet Bytes panes, noting the Wireshark colour code of each protocol.

Requirements

Windows laptop with Wireshark (with Npcap); active Wi-Fi connection; Command Prompt (for ping); a web browser (Brave/Chrome/Edge).

Capture Details

Parameter	Value used
Capture interface	Wi-Fi
Host (laptop) IP address	192.168.1.8
Host MAC address	14:5a:41:da:9a:33 (shown by Wireshark as CloudNetwork_da:9a:33)
Gateway (router) IP address	192.168.1.1
Gateway MAC address	44:95:3b:c6:c7:80 (shown as RLTechIndiaP_c6:c7:80)
Ping targets	192.168.1.1 (local router) and 1.1.1.1 (Internet)
Websites visited	www.bigbasket.com (HTTPS) and httpforever.com (plain HTTP)
Display filters used	http, dns, arp, icmp, tcp.flags.syn, tcp.flags.syn==1 && tcp.flags.ack==0, tcp.flags.syn==1 && tcp.flags.ack==1, tcp.flags.syn==0 && tcp.flags.ack==1, udp

General Steps for Any Wireshark Practical

Whatever the aim of the experiment (analyse HTTP, find a DNS query, study the TCP handshake, etc.), the workflow below can be followed every time.

1. **Read the aim and decide the traffic needed.** Note which protocol(s) must be shown (e.g. ICMP → needs ping; HTTP → needs an http:// site; DNS → any new domain name).
2. **Prepare the system.** Close unnecessary applications and browser tabs so that the capture is clean. Note your own IP and MAC address with the command ipconfig /all.
3. **Open Wireshark and select the correct interface** (Wi-Fi or Ethernet) – the one that shows a live activity graph. Optionally set a capture filter (e.g. host 192.168.1.1) to restrict what is recorded.
4. **Start the capture** with the blue shark-fin button (Ctrl+E).
5. **Generate the traffic.** Run ping, nslookup or tracert in Command Prompt and visit the required websites. Note the order and time of each action – it helps to locate packets later.

6. **Stop the capture** with the red square button (Ctrl+E) and save the file (File → Save As, .pcapng).
7. **Check the colouring rules** (View → Coloring Rules) so that every colour in the Packet List can be explained.
8. **Apply display filters** in the green filter bar (http, dns, arp, icmp, tcp, udp, ip.addr == x.x.x.x …) and press Enter. A green bar means the filter syntax is valid; red means an error.
9. **Select a packet and analyse it.** In the Packet Details pane expand the layers one by one – Frame → Ethernet II → Internet Protocol → TCP/UDP → application protocol – and note the important fields. The Packet Bytes pane shows the same packet in hexadecimal.
10. **Use extra tools when needed:** Follow → TCP Stream, Statistics → Conversations / Protocol Hierarchy, or double-click a packet to open it in a separate window.
11. **Take screenshots** that show the filter bar, the packet list and the expanded details together, and keep them in the order in which they were captured.
12. **Write the observations, result and conclusion** in the journal, and prepare the viva questions.

Commonly used display filters

Filter	Shows
http	HTTP requests and responses (port 80, unencrypted)
dns	DNS queries and responses (port 53)
arp	ARP requests and replies (IP → MAC mapping)
icmp	Ping (echo request / reply) and ICMP error messages
tcp / udp	All TCP or all UDP traffic
tcp.flags.syn==1 && tcp.flags.ack==0	Only SYN packets – the first step of the TCP three-way handshake
tcp.flags.syn==1 && tcp.flags.ack==1	Only SYN-ACK packets – the second step of the handshake
ip.addr == 192.168.1.8	All traffic to or from one host
tcp.port == 443	HTTPS (TLS) traffic

Wireshark Colour Codes (Default Coloring Rules)

Wireshark colours every row of the Packet List according to the Coloring Rules (View → Coloring Rules). The rules are checked from top to bottom and the first rule that matches decides the colour. The table below lists the default colours for the protocols studied in this experiment; the hex values were checked against the Coloring Rules window captured in Fig 5.

Protocol / Rule	Display filter used by the rule	Background	Text	Appearance
HTTP	http || tcp.port == 80 || http2	#E4FFC7	#12272E	Light green
TCP	tcp	#E7E6FF	#12272E	Light lavender
UDP	udp	#DAEEFF	#12272E	Light blue
ARP	arp	#FAF0D7	#12272E	Pale yellow / cream
ICMP	icmp || icmpv6	#FCE0FF	#12272E	Light pink
DNS	(no separate default rule)	#DAEEFF	#12272E	Light blue (as UDP)
TCP SYN/FIN	tcp.flags & 0x02 || tcp.flags.fin == 1	#A0A0A0	#12272E	Grey

Protocol / Rule	Display filter used by the rule	Background	Text	Appearance
TCP RST	tcp.flags.reset eq 1	#A40000	#FFFC9C	Dark red, yellow text
ICMP errors	icmp.type in {3..5, 11}	#12272E	#A1FF9C	Dark teal, green text

Note on DNS: the default rule list has no separate DNS rule, so a DNS packet takes the colour of the transport protocol it travels over – light blue when it is carried by UDP (the normal case) and lavender when it is carried by TCP. In this capture the router answered DNS over TCP port 53, so the DNS rows appear lavender (see Figs 7 and 8).

Sequence of Work

The experiment was done in two phases: first the traffic was captured and then, after stopping the capture, the packets were inspected. The figures in this journal are arranged in the same order in which the screenshots were taken.

Fig	Activity	Phase	Time (PM)
1	Ping to the gateway 192.168.1.1	Capture running	10:38
2	Ping to the Internet host 1.1.1.1	Capture running	10:38
3	Visited www.bigbasket.com (HTTPS)	Capture running	10:39
4	Visited httpforever.com (HTTP)	Capture running	10:39
5	Coloring Rules window	Capture stopped – inspection	10:42
6	HTTP packet (GET and 304 response)	Inspection	10:47
7 – 8	DNS query and DNS response	Inspection	10:48
9 – 10	ARP request and ARP reply	Inspection	10:51
11 – 12	ICMP echo request and echo reply	Inspection	10:52
13 – 16	TCP: all TCP, SYN, SYN-ACK, ACK/PSH-ACK	Inspection	10:58 – 11:00
17	UDP packet	Inspection	11:00

Procedure

A. Capturing the Traffic

Step 1: Open Wireshark and select the **Wi-Fi** interface (the one with the live activity graph). Click **Start capturing packets** (blue shark-fin icon).

Step 2: While the capture is running, open **Command Prompt** and ping the local gateway to generate ICMP traffic inside the LAN:

```
C:\Users\lachh> ping 192.168.1.1
```

All 4 echo requests received replies (time = 2–3 ms, TTL = 64, 0% loss). The TTL of 64 is the initial TTL of the router; the router is one hop away, so the value is not reduced.

```
C:\Users\lachh>ping 192.168.1.1

Pinging 192.168.1.1 with 32 bytes of data:
Reply from 192.168.1.1: bytes=32 time=3ms TTL=64
Reply from 192.168.1.1: bytes=32 time=2ms TTL=64
Reply from 192.168.1.1: bytes=32 time=3ms TTL=64
Reply from 192.168.1.1: bytes=32 time=2ms TTL=64

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 2ms, Maximum = 3ms, Average = 2ms

C:\Users\lachh>
```

Fig 1: ping 192.168.1.1 – 4 sent, 4 received, 0% loss, TTL=64

Step 3: Ping a host on the Internet to generate ICMP traffic that crosses the router:

```
C:\Users\lachh> ping 1.1.1.1
```

The replies took 7–11 ms (average 8 ms) with TTL = 56, with 0% loss. The TTL has dropped from the initial 64 to 56, i.e. the packets passed through 8 routers; the longer delay compared to the gateway is due to the Internet path.

```
C:\Users\lachh>ping 1.1.1.1

Pinging 1.1.1.1 with 32 bytes of data:
Reply from 1.1.1.1: bytes=32 time=8ms TTL=56
Reply from 1.1.1.1: bytes=32 time=11ms TTL=56
Reply from 1.1.1.1: bytes=32 time=7ms TTL=56
Reply from 1.1.1.1: bytes=32 time=7ms TTL=56

Ping statistics for 1.1.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 7ms, Maximum = 11ms, Average = 8ms

C:\Users\lachh>
```

Fig 2: ping 1.1.1.1 – 4 sent, 4 received, 0% loss, TTL=56

Step 4: Open a browser and visit **https://www.bigbasket.com**. The padlock in the address bar shows that the site is encrypted with HTTPS (TLS), so the page content cannot be read in Wireshark.



Fig 3: www.bigbasket.com opened over HTTPS (padlock)

Step 5: Visit **http://httpforever.com**. The browser shows **Not secure** because the site uses plain HTTP (port 80); its requests and responses are visible in Wireshark as HTTP packets.

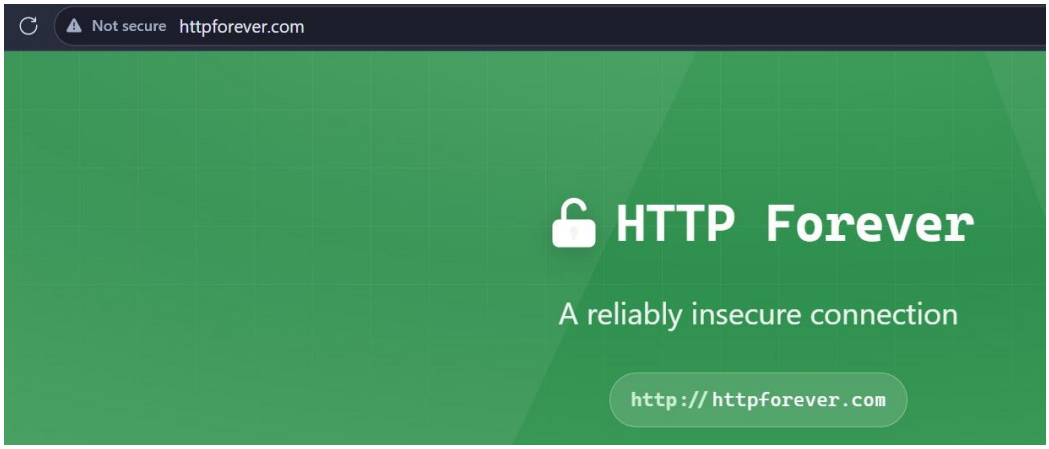


Fig 4: httpforever.com opened over HTTP – "Not secure"

Step 6: Return to Wireshark and click **Stop capturing packets** (red square). The capture is now complete and ready for inspection.

B. Inspecting the Captured Packets

Step 7: Open **View → Coloring Rules** to see which colour Wireshark uses for each protocol.

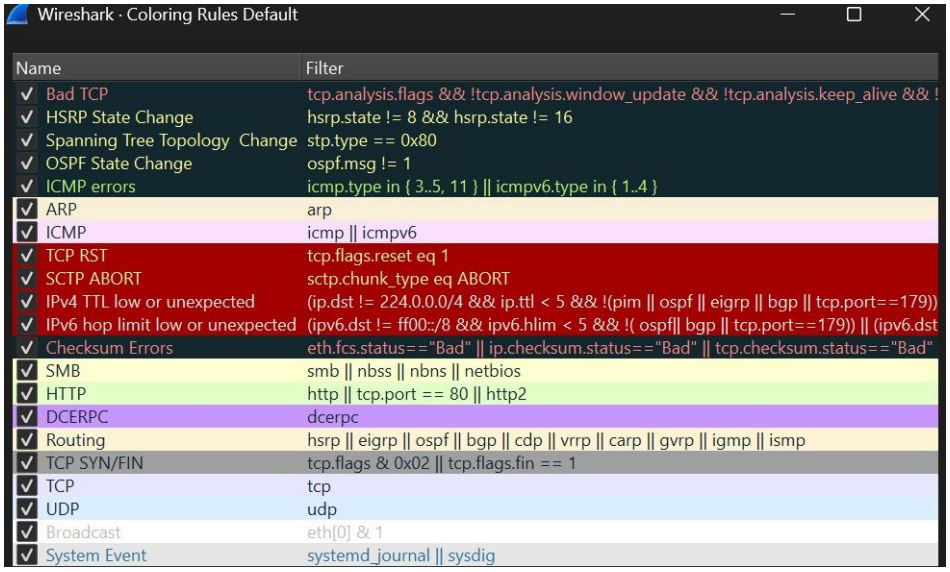


Fig 5: Wireshark – Coloring Rules (default)

From the Coloring Rules window: ARP is pale yellow (#FAF0D7), ICMP is light pink (#FCE0FF), HTTP is light green (#E4FFC7), TCP is light lavender (#E7E6FF), UDP is light blue (#DAEEFF), TCP SYN/FIN is grey (#A0A0A0) and TCP RST is dark red (#A40000) with yellow text. These colours are used in all the following screenshots.

1. HTTP Analysis

Step 8: Type **http** in the display filter bar and press Enter. Only two packets are shown: the request sent by the laptop and the response from the server. Double-click packet 40784 to open it in its own window and expand the Transmission Control Protocol section.

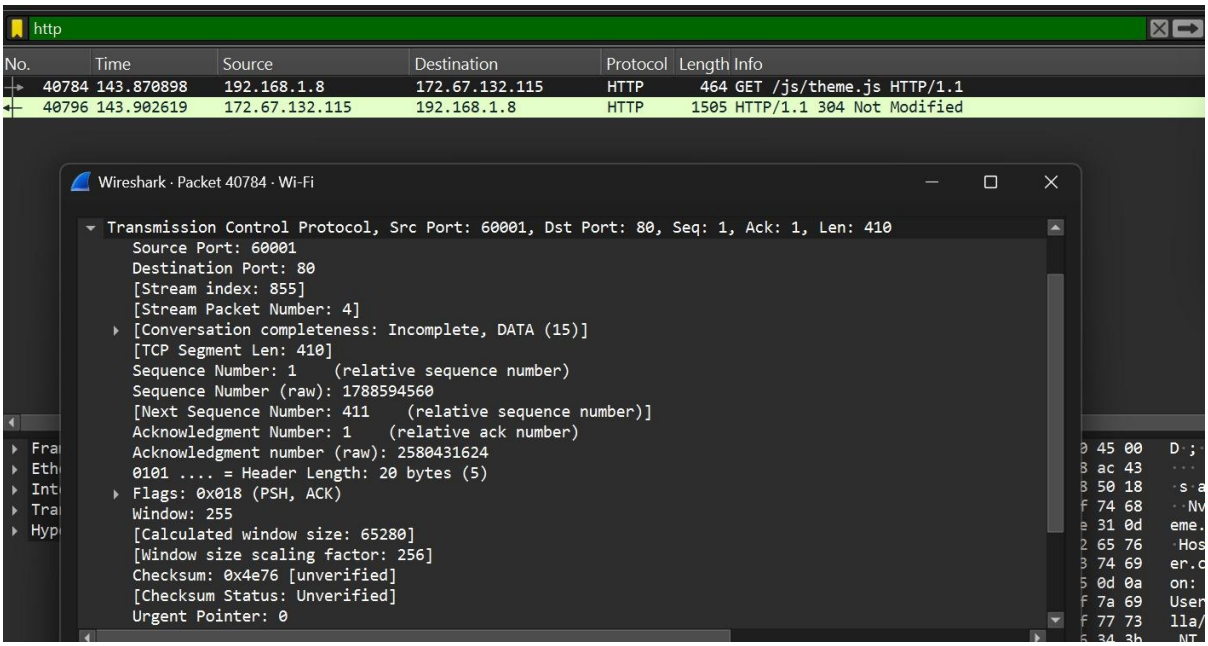


Fig 6: Display filter "http" – GET /js/theme.js and 304 Not Modified; TCP details of packet 40784

Packets in the list

Field	Observation
No. 40784	t = 143.870898 s, 192.168.1.8 → 172.67.132.115, HTTP, 464 bytes – **GET /js/theme.js HTTP/1.1** (request)
No. 40796	t = 143.902619 s, 172.67.132.115 → 192.168.1.8, HTTP, 1505 bytes – **HTTP/1.1 304 Not Modified** (response, shown in light green)
Response time	143.902619 − 143.870898 ≈ 31.7 ms

TCP layer of packet 40784

Field	Observation
Source port / Destination port	60001 / 80 (port 80 = HTTP)
Stream index	855
Sequence number	1 (relative); raw 1788594560
Next sequence number	411 (1 + 410 bytes of data)
Acknowledgment number	1 (relative); raw 2580431624
TCP segment length	410 bytes (the HTTP request itself)
Header length	20 bytes (5 words)
Flags	0x018 – PSH, ACK
Window size	255; calculated window size 65280 (scaling factor 256)
Checksum	0x4e76 [unverified]

Field	Observation
Urgent pointer	0

Analysis:

- The laptop asks for the file **/js/theme.js** of httpforever.com using the GET method. The server (172.67.132.115, a Cloudflare address) replies **304 Not Modified**, meaning the copy cached in the browser is still valid, so the file is not sent again.
- The request is carried in a TCP segment with PSH+ACK flags: PSH asks the receiver to pass the data to the application at once, and ACK acknowledges the data received so far.
- Since the destination port is 80 and the data is not encrypted, the request can be read directly in Wireshark. The HTTPS site (bigbasket) would show only TLS "Application Data" on port 443.
- Colour: the HTTP response row is light green (#E4FFC7), as per the HTTP rule.

2. DNS Analysis

Step 9: Apply the filter **dns**. When the browser opened httpforever.com, the laptop (192.168.1.8) asked the router/DNS server (192.168.1.1) for the address of the site. Double-click the query, packet 40740.

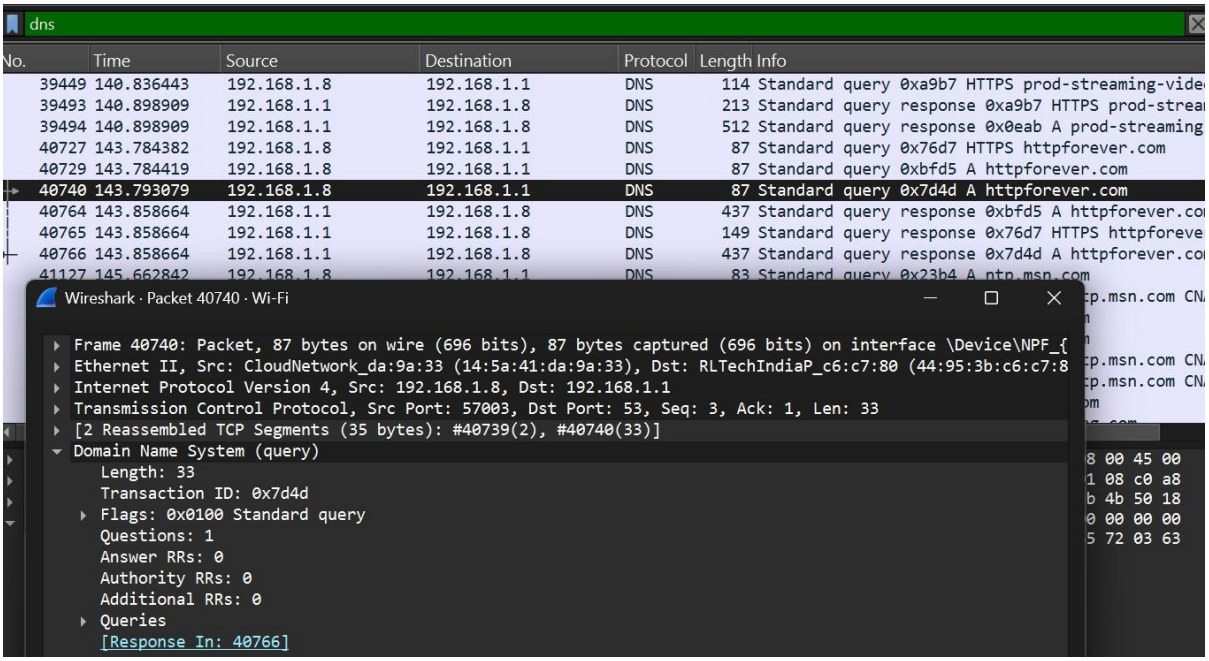


Fig 7: Display filter "dns" – DNS query for httpforever.com (packet 40740)

DNS query – packet 40740

Field	Observation
Time / Length	143.793079 s; 87 bytes on wire (696 bits)
Ethernet II	Src: CloudNetwork_da:9a:33 (14:5a:41:da:9a:33) → Dst: RLTechIndiaP_c6:c7:80 (44:95:3b:c6:c7:80)
IPv4	Src: 192.168.1.8 → Dst: 192.168.1.1
TCP	Src port 57003 → Dst port 53, Seq 3, Ack 1, Len 33
Reassembly	2 reassembled TCP segments (35 bytes): #40739 (2 bytes) + #40740 (33 bytes)
DNS length	33
Transaction ID	0x7d4d

Field	Observation
Flags	0x0100 – Standard query (recursion desired)
Questions / Answer / Authority / Additional	1 / 0 / 0 / 0
Response In	Frame 40766

Analysis:

- The laptop sent three queries for httpforever.com at about 143.784–143.793 s: 0x76d7 (HTTPS record), 0xbfd5 (A record) and 0x7d4d (A record). A query has Answer RRs = 0 because the answer is still to be given.
- The query uses **TCP port 53** instead of the usual UDP. In DNS over TCP every message is preceded by a 2-byte length field, which is why Wireshark reassembles 2 bytes + 33 bytes = 35 bytes.
- Because of the TCP transport, the rows are drawn in the TCP colour (lavender) and not in UDP light blue.
- The Transaction ID is used to match a response with its query.

Step 10: Select the matching response, packet **40766** (same Transaction ID), and open it.

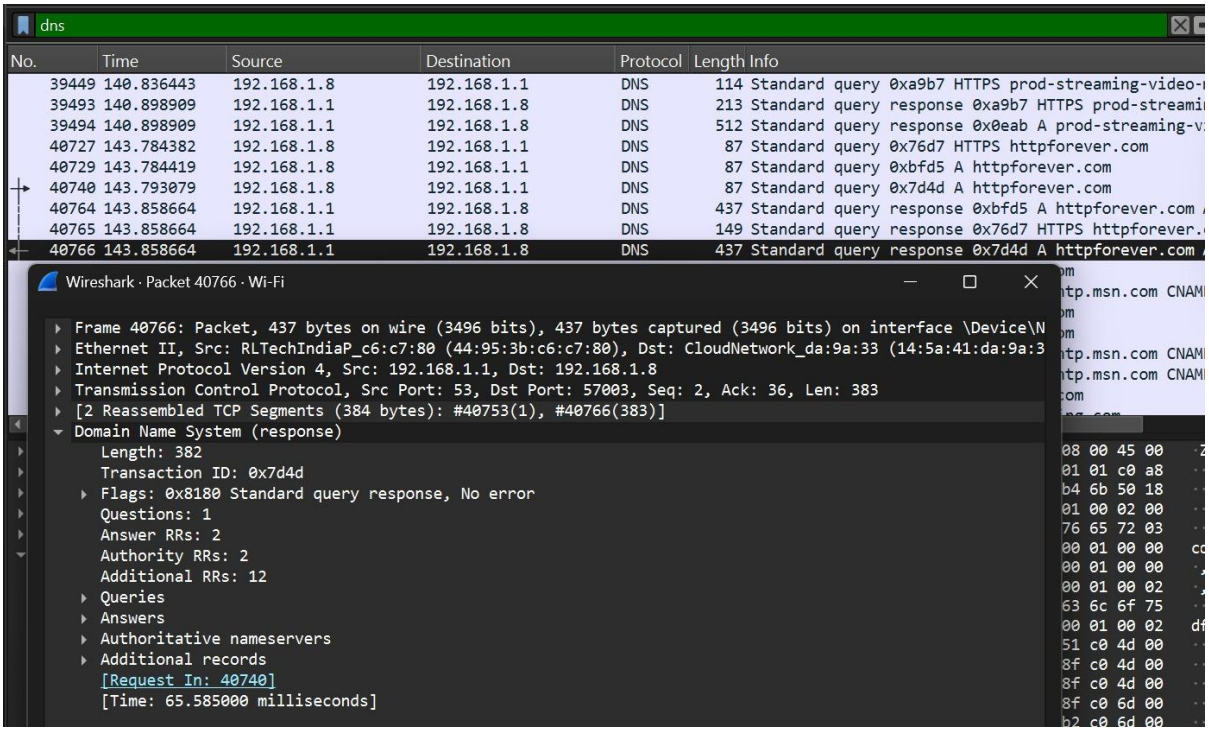


Fig 8: DNS response for httpforever.com (packet 40766)

DNS response – packet 40766

Field	Observation
Time / Length	143.858664 s; 437 bytes on wire (3496 bits)
Ethernet II	Src: RLTechIndiaP_c6:c7:80 (44:95:3b:c6:c7:80) → Dst: CloudNetwork_da:9a:33 (14:5a:41:da:9a:33)
IPv4	Src: 192.168.1.1 → Dst: 192.168.1.8
TCP	Src port 53 → Dst port 57003, Seq 2, Ack 36, Len 383
Reassembly	2 reassembled TCP segments (384 bytes): #40753 (1 byte) + #40766 (383 bytes)

Field	Observation
DNS length	382
Transaction ID	0x7d4d (same as the query)
Flags	0x8180 – Standard query response, No error
Questions / Answer / Authority / Additional	1 / 2 / 2 / 12
Request In / Time	Frame 40740 / 65.585 ms

Analysis:

- The response carries the same Transaction ID (0x7d4d), the source and destination (and the ports) are swapped, and the flags 0x8180 show that it is a response with “No error”.
- The response gave 2 answer records (the IP addresses of httpforever.com), 2 authoritative name servers and 12 additional records.
- The DNS lookup took 143.858664 − 143.793079 = **65.585 ms**, after which the browser could open the TCP connection and send the HTTP GET request (Fig 6).
- Other DNS rows in the list (e.g. ntp.msn.com, prod-streaming-video…) are background requests made by other programs on the laptop.

3. ARP Analysis

Step 11: Apply the filter **arp**. Three request/reply pairs appear (frames 1890/1891, 8354/8355 and 11798/11799). Select the first request, frame 1890, and expand the Address Resolution Protocol section.

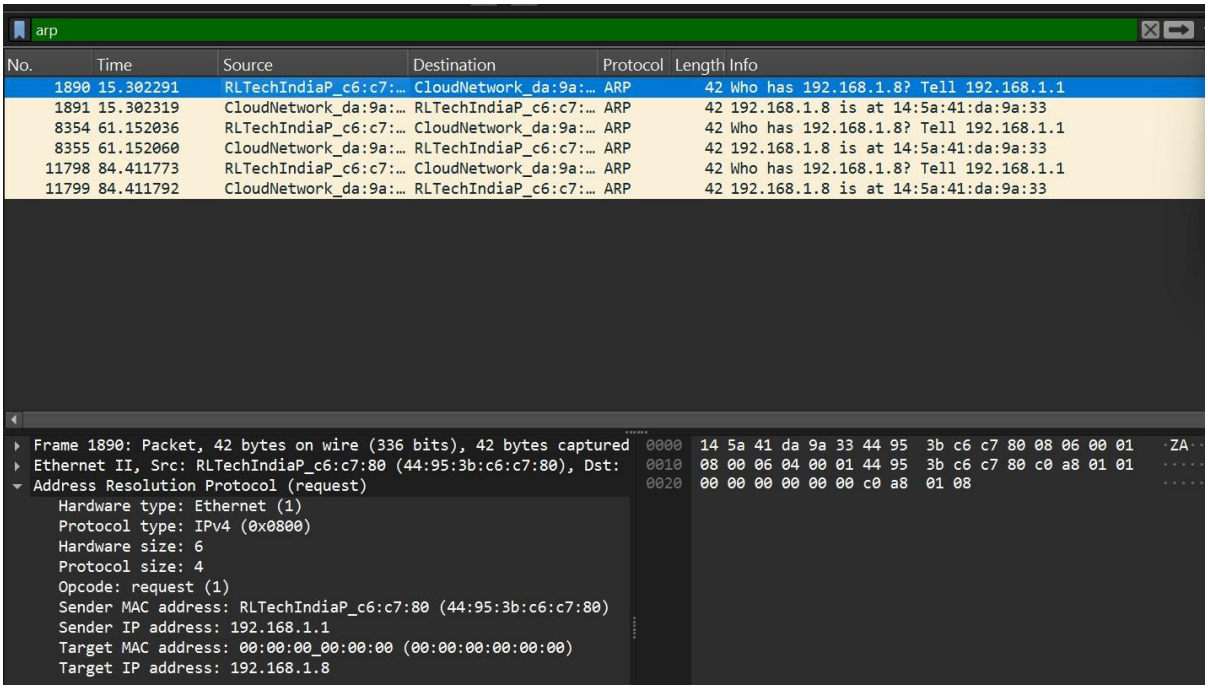


Fig 9: Display filter “arp” – ARP request (frame 1890: Who has 192.168.1.8? Tell 192.168.1.1)

ARP request – frame 1890

Field	Observation
Time / Length	15.302291 s; 42 bytes on wire (336 bits)
Info	Who has 192.168.1.8? Tell 192.168.1.1
Hardware type	Ethernet (1)

Field	Observation
Protocol type	IPv4 (0x0800)
Hardware size / Protocol size	6 / 4
Opcode	request (1)
Sender MAC address	RLTechIndiaP_c6:c7:80 (44:95:3b:c6:c7:80)
Sender IP address	192.168.1.1
Target MAC address	00:00:00_00:00:00 (00:00:00:00:00:00) – unknown
Target IP address	192.168.1.8

Step 12: Select the next packet, frame **1891**, which is the reply to this request.

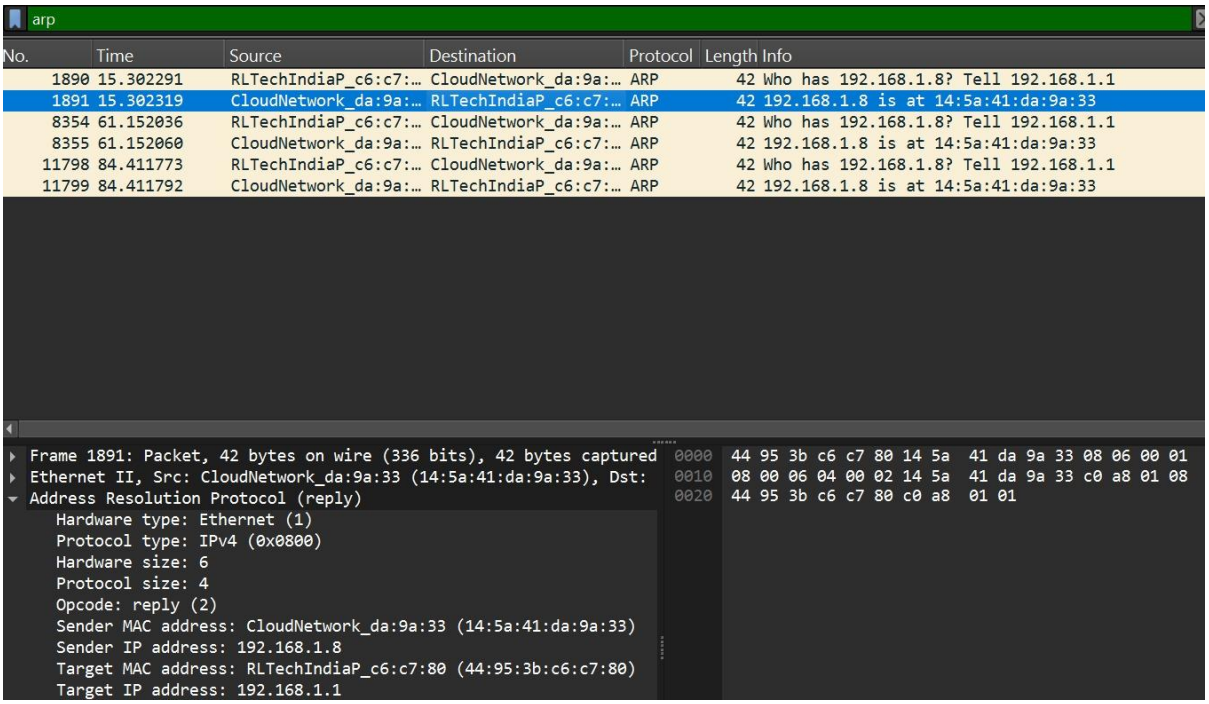


Fig 10: ARP reply (frame 1891: 192.168.1.8 is at 14:5a:41:da:9a:33)

ARP reply – frame 1891

Field	Observation
Time / Length	15.302319 s; 42 bytes on wire (336 bits)
Info	192.168.1.8 is at 14:5a:41:da:9a:33
Opcode	reply (2)
Sender MAC / IP	CloudNetwork_da:9a:33 (14:5a:41:da:9a:33) / 192.168.1.8
Target MAC / IP	RLTechIndiaP_c6:c7:80 (44:95:3b:c6:c7:80) / 192.168.1.1

Analysis:

- ARP converts an IP address into a MAC address on the local network. Here the router (192.168.1.1) asks “Who has 192.168.1.8?” and the laptop answers with its MAC address 14:5a:41:da:9a:33.
- The target MAC of the request is 00:00:00:00:00:00 because it is not yet known. In this capture the request was sent directly (unicast) to the laptop’s MAC (the hex bytes begin with 14 5a 41 da

9a 33) – the router was refreshing an existing ARP entry. A first-time ARP request is normally sent to the broadcast address ff:ff:ff:ff:ff:ff.

- The reply came only 28 µs after the request (15.302319 − 15.302291), because both devices are on the same LAN.
- ARP packets have no IP header, so they are only 42 bytes long (14-byte Ethernet header + 28-byte ARP message). Colour: pale yellow (#FAF0D7).

4. ICMP Analysis

Step 13: Apply the filter **icmp**. The list shows the pings done in Steps 2 and 3: requests/replies with the gateway 192.168.1.1 (seq 57–60) followed by those with 1.1.1.1 (seq 61–64). Select the first echo request, frame 1483.

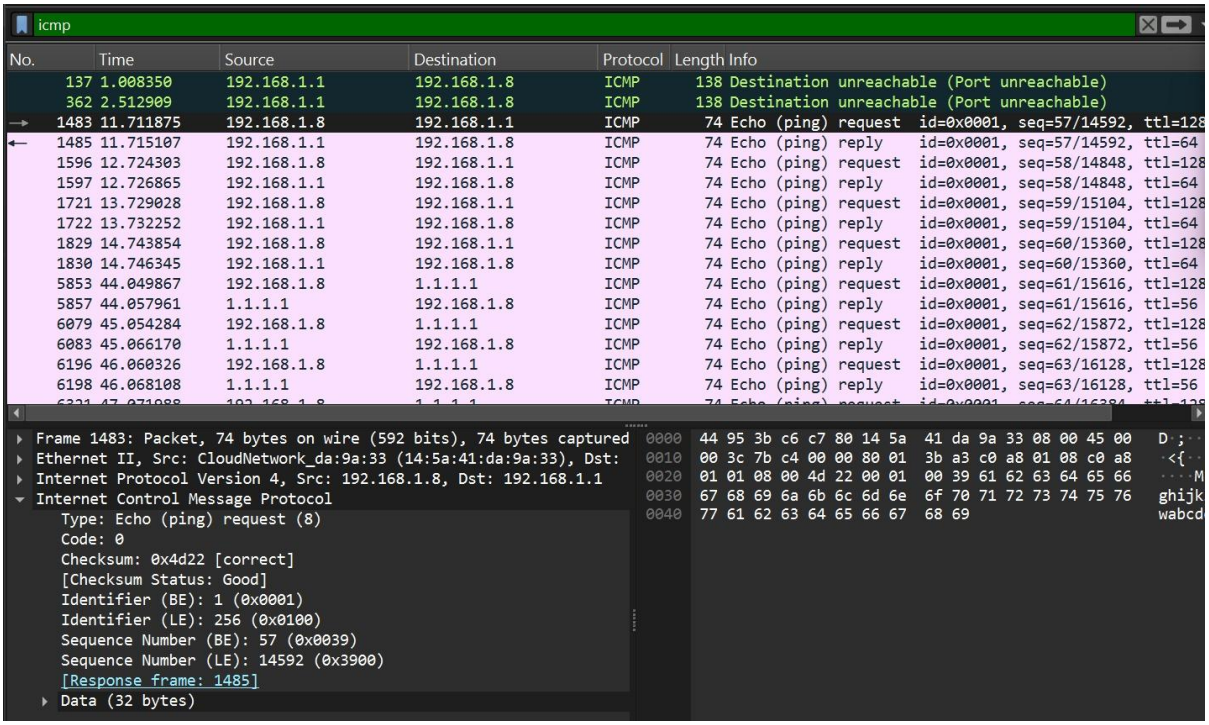


Fig 11: Display filter "icmp" – Echo (ping) request, frame 1483

ICMP echo request – frame 1483

Field	Observation
Time / Length	11.711875 s; 74 bytes on wire (592 bits)
Ethernet II	Src: CloudNetwork_da:9a:33 (14:5a:41:da:9a:33) → Dst: 44:95:3b:c6:c7:80 (router)
IPv4	Src: 192.168.1.8 → Dst: 192.168.1.1; TTL = 128
Type / Code	Echo (ping) request (8) / 0
Checksum	0x4d22 [correct]; Checksum Status: Good
Identifier	1 (0x0001) BE; 256 (0x0100) LE
Sequence number	57 (0x0039) BE; 14592 (0x3900) LE
Response frame	1485
Data	32 bytes (abcdefghijklmnopqrstuvwabcdefghi)

Step 14: Select the reply, frame **1485**.

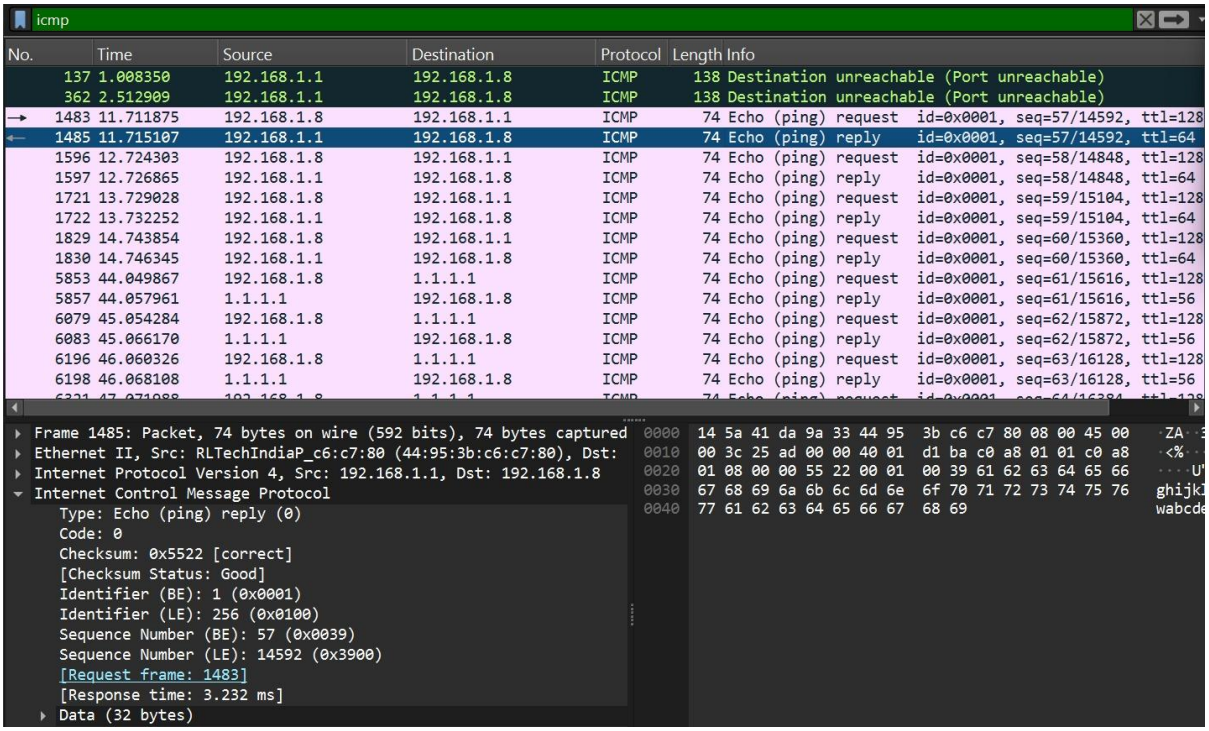


Fig 12: ICMP Echo (ping) reply, frame 1485 – response time 3.232 ms

ICMP echo reply – frame 1485

Field	Observation
Time / Length	11.715107 s; 74 bytes on wire (592 bits)
Ethernet II	Src: RLTechIndiaP_c6:c7:80 (44:95:3b:c6:c7:80) → Dst: CloudNetwork_da:9a:33
IPv4	Src: 192.168.1.1 → Dst: 192.168.1.8; TTL = 64
Type / Code	Echo (ping) reply (0) / 0
Checksum	0x5522 [correct]; Checksum Status: Good
Identifier / Sequence number	1 (0x0001) / 57 (0x0039) – same as the request
Request frame	1483
Response time	3.232 ms
Data	32 bytes (same as the request)

Analysis:

- ICMP is used by ping: type 8 is an echo request and type 0 is an echo reply. The reply copies the identifier, sequence number and data of the request, which is how Wireshark pairs them (Response frame / Request frame).
- The response time is 11.715107 − 11.711875 = **3.232 ms**, matching the 2–3 ms shown by ping in Fig 1.
- The TTL of the request is 128 (Windows default); the reply from the router has TTL 64, and replies from 1.1.1.1 have TTL 56 (see the list) – the same values as seen in Figs 1 and 2.
- Two ICMP “Destination unreachable (Port unreachable)” packets (frames 137 and 362, 138 bytes, from 192.168.1.1 to 192.168.1.8) are also visible at the top of the list. They are ICMP errors and so use the dark ICMP-errors colour; normal ICMP rows are light pink (#FCE0FF).

5. TCP Analysis

Step 15: Apply the filter **tcp.flags.syn**. This filter simply tests whether the TCP flags field exists, so every TCP packet is listed (including TLSv1.2 “Application Data” packets that run on TCP). Select packet 796 and look at its TCP header.

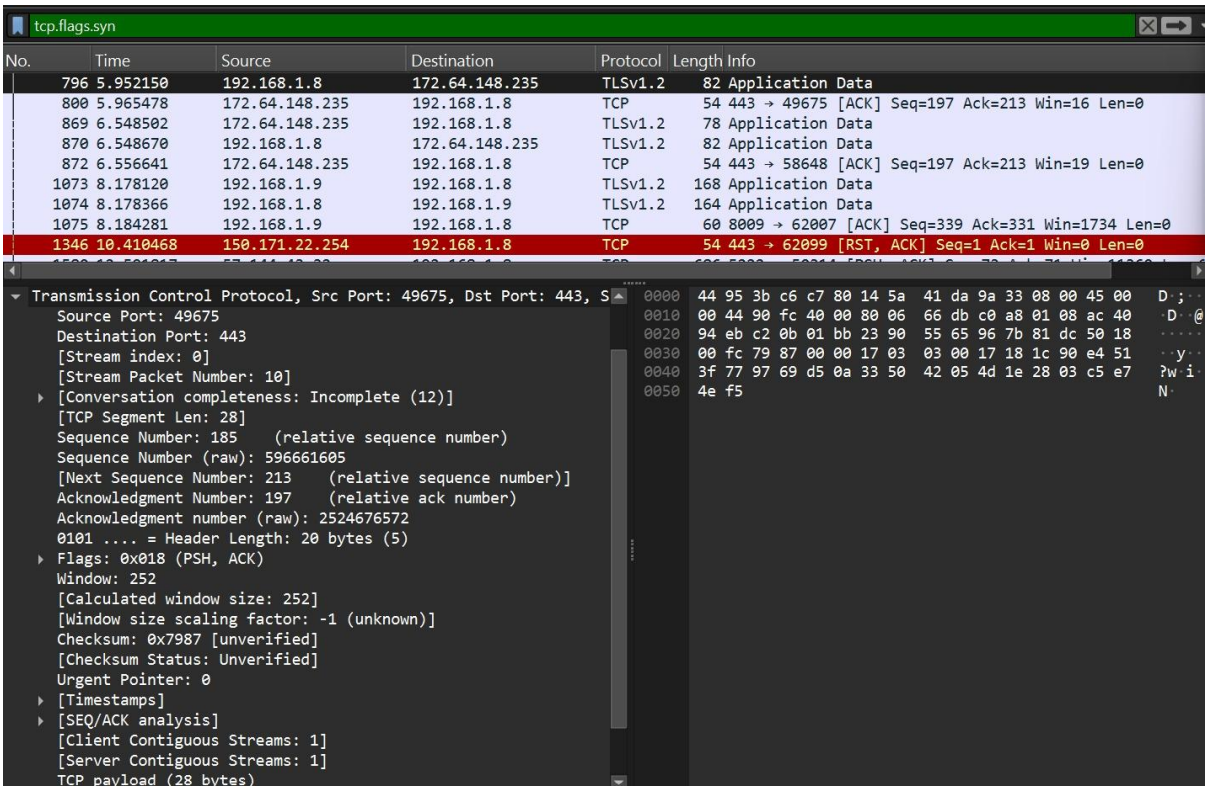


Fig 13: Display filter “tcp.flags.syn” – TCP packets and the TCP header of packet 796

TCP header of packet 796

Field	Observation
Packet	No. 796, t = 5.952150 s, 192.168.1.8 → 172.64.148.235, TLSv1.2 “Application Data”, 82 bytes
Source port / Destination port	49675 / 443 (HTTPS)
Stream index / Stream packet no.	0 / 10
Conversation completeness	Incomplete (12)
TCP segment length	28 bytes
Sequence number	185 (relative); raw 596661605
Next sequence number	213 (185 + 28)
Acknowledgment number	197 (relative); raw 2524676572
Header length	20 bytes (5)
Flags	0x018 – PSH, ACK
Window size	252 (calculated 252); scaling factor −1 (unknown)
Checksum	0x7987 [unverified]

Analysis:

- The list shows several colours: light lavender (#E7E6FF) for normal TCP/TLS packets and a **dark red row with yellow text** (frame 1346: 150.171.22.254 → 192.168.1.8, port 443 → 62099, [RST, ACK]). This is the "TCP RST" rule – the server abruptly reset a connection.
- The window-scaling factor is "unknown" and the conversation is "Incomplete" because this connection was already open when the capture started, so its SYN handshake was not captured.
- Packets 1073–1075 are between the laptop and another LAN device, 192.168.1.9 (port 8009, used by Google Cast devices).

Step 16: To look at the **three-way handshake**, first show only the SYN packets (first step) with the filter **tcp.flags.syn==1 && tcp.flags.ack==0**. Select frame 3184.

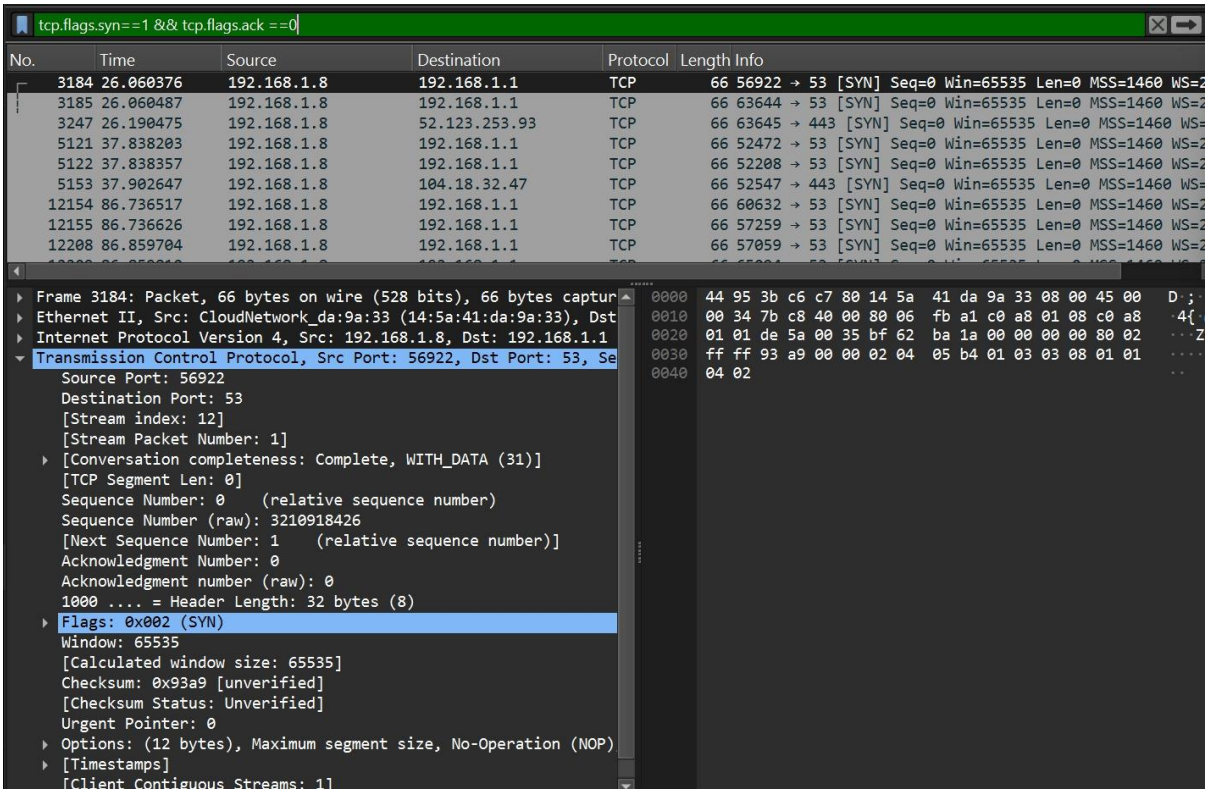


Fig 14: Filter "tcp.flags.syn==1 && tcp.flags.ack==0" – SYN packets, frame 3184

TCP SYN – frame 3184

Field	Observation
Time / Length	26.060376 s; 66 bytes on wire (528 bits)
Ethernet II	Src: CloudNetwork_da:9a:33 → Dst: RLTechIndiaP_c6:c7:80
IPv4	Src: 192.168.1.8 → Dst: 192.168.1.1
Source port / Destination port	56922 / 53
Stream index / Stream packet no.	12 / 1
Conversation completeness	Complete, WITH_DATA (31)
TCP segment length	0
Sequence number	0 (relative); raw 3210918426 (initial sequence number)
Next sequence number	1
Acknowledgment number	0

Field	Observation
Header length	32 bytes (8) – 20 bytes + 12 bytes of options
Flags	0x002 – SYN
Window size	65535
Checksum	0x93a9 [unverified]
Options (12 bytes)	Maximum segment size (MSS = 1460), No-Operation (NOP), window scale (WS = 256), SACK permitted

Step 17: Now show the second step of the handshake with the filter **tcp.flags.syn==1 && tcp.flags.ack==1**. Select frame 3186.

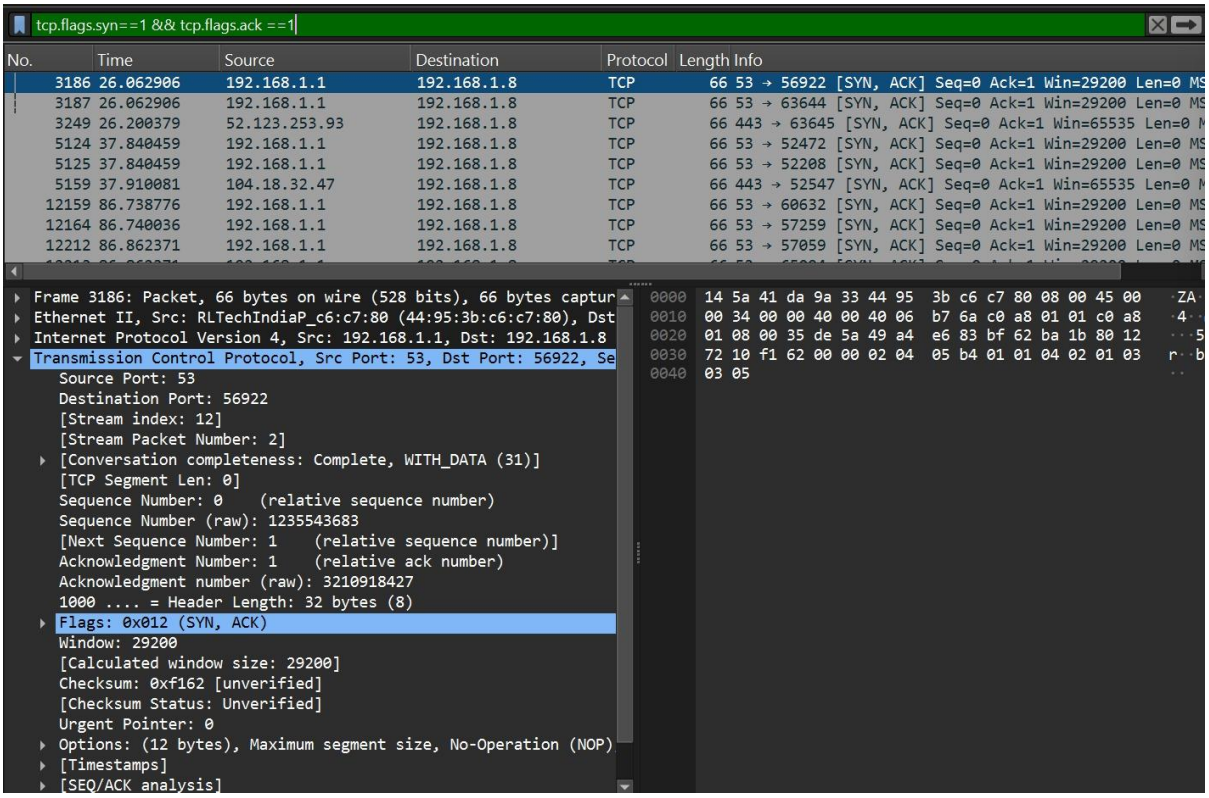


Fig 15: Filter "tcp.flags.syn==1 && tcp.flags.ack==1" – SYN-ACK packets, frame 3186

TCP SYN-ACK – frame 3186

Field	Observation
Time / Length	26.062906 s; 66 bytes on wire (528 bits)
Ethernet II	Src: RLTechIndiaP_c6:c7:80 → Dst: CloudNetwork_da:9a:33
IPv4	Src: 192.168.1.1 → Dst: 192.168.1.8
Source port / Destination port	53 / 56922
Stream index / Stream packet no.	12 / 2
Sequence number	0 (relative); raw 1235543683
Acknowledgment number	1 (relative); raw 3210918427
Header length	32 bytes (8)

Field	Observation
Flags	0x012 – SYN, ACK
Window size	29200
Checksum	0xf162 [unverified]

Step 18: Finally, apply **tcp.flags.syn==0 && tcp.flags.ack==1** to see the acknowledgement/data packets after the handshake. Select frame 3190.

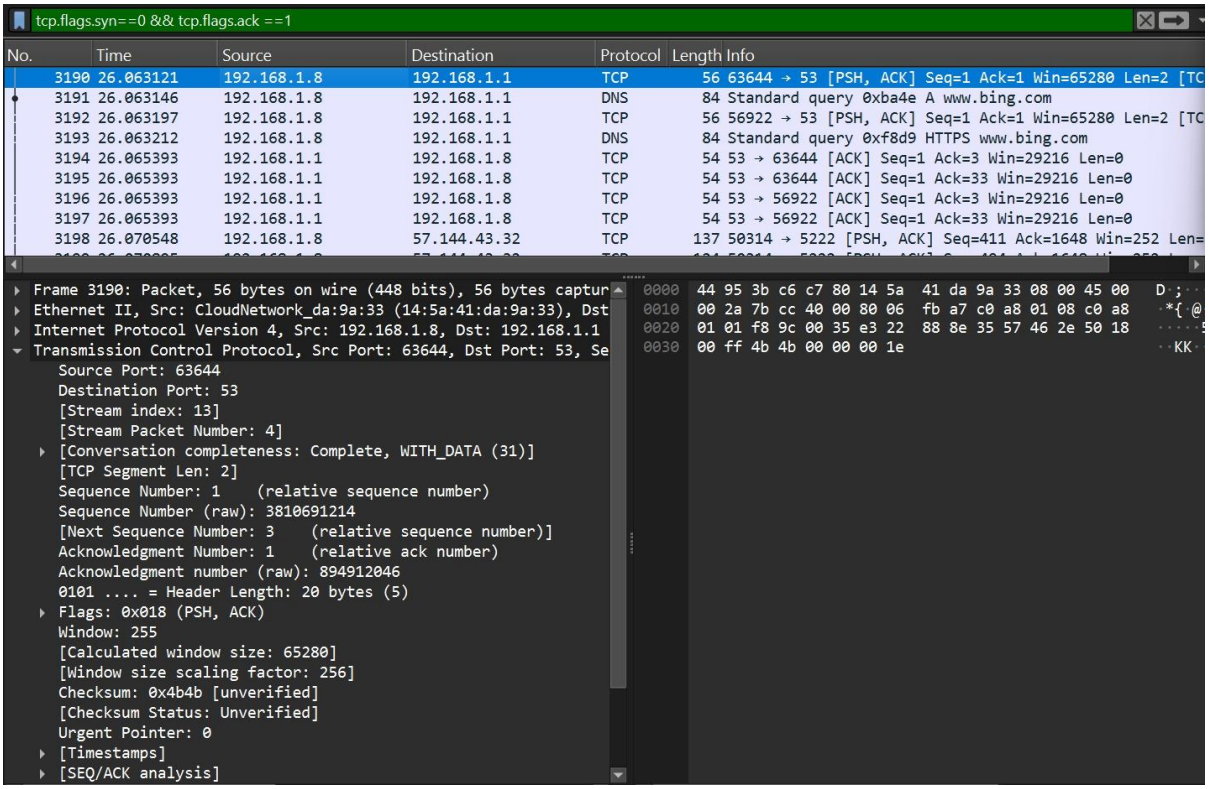


Fig 16: Filter "tcp.flags.syn==0 && tcp.flags.ack==1" – ACK / PSH-ACK packets, frame 3190

TCP PSH, ACK – frame 3190

Field	Observation
Time / Length	26.063121 s; 56 bytes on wire (448 bits)
Ethernet II / IPv4	CloudNetwork_da:9a:33 → RLTechIndiaP_c6:c7:80; 192.168.1.8 → 192.168.1.1
Source port / Destination port	63644 / 53
Stream index / Stream packet no.	13 / 4
TCP segment length	2
Sequence number	1 (relative); raw 3810691214
Next sequence number	3
Acknowledgment number	1 (relative); raw 894912046
Header length	20 bytes (5)
Flags	0x018 – PSH, ACK

Field	Observation
Window size	255; calculated window size 65280 (scaling factor 256)
Checksum	0x4b4b [unverified]

Analysis:

- **Three-way handshake (stream 12):** (1) the laptop sends **SYN** with ISN = 3210918426 (frame 3184); (2) the router replies **SYN, ACK** with its own ISN = 1235543683 and Ack = 3210918427, i.e. the laptop's ISN + 1 (frame 3186); (3) the laptop completes the connection with an **ACK**. The SYN-ACK came 2.53 ms after the SYN.
- After the handshake, data is sent with **PSH, ACK** segments, as in frame 3190, and the server acknowledges it with plain ACK packets (frames 3194–3197 show Ack = 3 and Ack = 33). The sequence number of data grows by the number of bytes sent: Seq = 1, Len = 2 → next Seq = 3.
- The SYN rows are grey (#A0A0A0, "TCP SYN/FIN" rule), while the other TCP rows are lavender (#E7E6FF). Frames 3191 and 3193 (DNS queries for www.bing.com, a background request) are carried inside these TCP segments, as the 2-byte length prefix explains the Len = 2 segment.
- The SYN packets to 52.123.253.93 and 104.18.32.47 (port 443) in the list are the handshakes of HTTPS connections, while the many SYNs to 192.168.1.1 port 53 are DNS-over-TCP connections to the router.

6. UDP Analysis

Step 19: Apply the filter **udp**. Select packet 3177 and expand the User Datagram Protocol section.

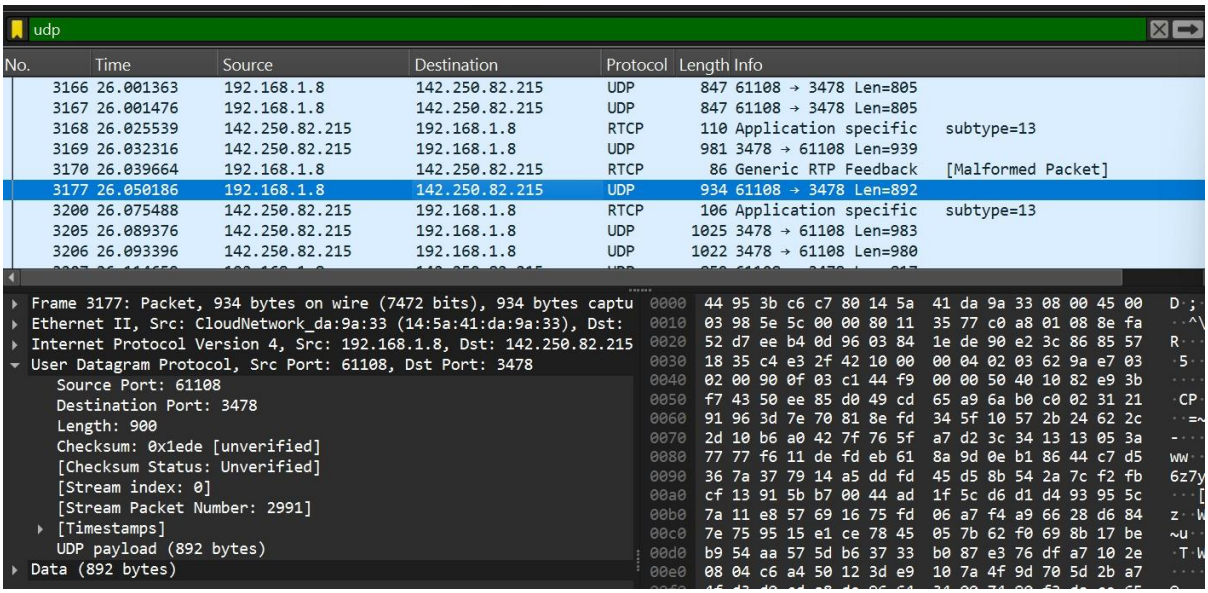


Fig 17: Display filter "udp" – UDP datagram, frame 3177 (port 61108 → 3478)

UDP datagram – frame 3177

Field	Observation
Time / Length	26.050186 s; 934 bytes on wire (7472 bits)
Ethernet II	Src: CloudNetwork_da:9a:33 → Dst: RLTechIndiaP_c6:c7:80
IPv4	Src: 192.168.1.8 → Dst: 142.250.82.215; TTL 128; protocol 17 (UDP)
Source port / Destination port	61108 / 3478
UDP length	900 bytes

Field	Observation
Checksum	0x1ede [unverified]
Stream index / Stream packet no.	0 / 2991
UDP payload	892 bytes

Analysis:

- The UDP header has only four fields – source port, destination port, length and checksum – and is just **8 bytes** long, against 20–32 bytes for TCP. UDP length 900 = 8-byte header + 892 bytes of payload.
- There is no handshake, sequence number, acknowledgment or window field in UDP: the datagrams are simply sent (connectionless, unreliable but fast). That is why it is used for real-time traffic.
- Destination port 3478 is the STUN/TURN port used by real-time audio/video (WebRTC) applications and the surrounding RTCP rows ("Application specific", "Generic RTP Feedback") show that a live media call/stream was running to a Google server (142.250.82.215) during the capture.
- UDP rows are light blue (#DAEEFF). Frame size check: 14 (Ethernet) + 20 (IP) + 8 (UDP) + 892 (payload) = 934 bytes.

Summary of Observations

Protocol	Colour	Layer / Port	Key observation
HTTP	Light green	Application / TCP 80	GET /js/theme.js answered by 304 Not Modified; readable because it is not encrypted
TCP	Lavender; SYN grey; RST red	Transport	Three-way handshake SYN → SYN-ACK → ACK; PSH-ACK carries data; RST resets a connection
UDP	Light blue	Transport	8-byte header, connectionless; 61108 → 3478 (STUN/WebRTC)
DNS	Follows transport (lavender here)	Application / port 53	Query 0x7d4d for httpforever.com answered in 65.585 ms
ARP	Pale yellow	Link layer	"Who has 192.168.1.8?" → "192.168.1.8 is at 14:5a:41:da:9a:33"
ICMP	Light pink	Network layer	Echo request (type 8) → echo reply (type 0), 3.232 ms; TTL 128 / 64 / 56

Result

✓ EXPERIMENT COMPLETED SUCCESSFULLY

Network traffic was captured with Wireshark on the Wi-Fi interface while pinging 192.168.1.1 and 1.1.1.1 and browsing www.bigbasket.com and httpforever.com. After stopping the capture, HTTP, DNS, ARP, ICMP, TCP and UDP packets were filtered and analysed, their fields were studied, and the colour code of each protocol in the Wireshark Coloring Rules was noted.

VIVA QUESTIONS AND ANSWERS

A. Wireshark Basics

Q1. What is Wireshark?

Ans: Wireshark is a free, open-source network protocol analyser. It captures packets passing through a network interface and shows them in detail, layer by layer, so that traffic can be studied and problems diagnosed.

Q2. What is a packet sniffer?

Ans: A packet sniffer is a program that captures and records the packets travelling on a network interface. Wireshark is a sniffer with a graphical interface and powerful filtering and decoding of protocols.

Q3. What are the three main panes of Wireshark?

Ans: The Packet List (one line per packet), the Packet Details (the selected packet decoded layer by layer) and the Packet Bytes (the raw packet in hexadecimal and ASCII).

Q4. What is the difference between a capture filter and a display filter?

Ans: A capture filter (BPF syntax, e.g. host 192.168.1.1) is applied before capture and decides which packets are recorded. A display filter (e.g. http, ip.addr == 192.168.1.8) is applied after capture and only hides packets in the view, without deleting them.

Q5. What does the green or red filter bar mean?

Ans: Green means the display filter syntax is valid; red means that the filter is wrong; yellow is a warning that the filter may not do what is expected.

Q6. What is a .pcapng file?

Ans: It is the default Wireshark capture file format (pcap next generation) used to save and reopen captured packets, including interface and comment information.

Q7. What is Npcap?

Ans: Npcap is the packet-capture driver/library for Windows that Wireshark uses to read packets from the network adapter.

Q8. What are the colour codes in Wireshark for?

Ans: The Coloring Rules colour each packet row by protocol or condition so that traffic types can be recognised quickly – e.g. green for HTTP, lavender for TCP, light blue for UDP, pale yellow for ARP, pink for ICMP, grey for TCP SYN/FIN and dark red for TCP RST.

Q9. What is promiscuous mode?

Ans: It is an adapter mode in which the NIC passes up all frames it sees, not only those addressed to it. On Wi-Fi, packets of other devices are generally not seen unless monitor mode is available.

B. Protocol Questions

Q10. What is the colour of HTTP, TCP, UDP, ARP and ICMP packets by default?

Ans: HTTP – light green (#E4FFC7); TCP – lavender (#E7E6FF); UDP – light blue (#DAEEFF); ARP – pale yellow (#FAF0D7); ICMP – light pink (#FCE0FF). DNS has no rule of its own and takes the colour of UDP or TCP.

Q11. What is the use of ARP?

Ans: ARP (Address Resolution Protocol) finds the MAC address that belongs to an IP address on the local network. A host broadcasts “Who has IP x?” and the owner replies “IP x is at MAC y”.

Q12. What is DNS and which port does it use?

Ans: The Domain Name System converts domain names such as httpforever.com into IP addresses. It normally uses UDP port 53, and TCP port 53 for large responses or zone transfers (as seen in this capture).

Q13. What is ICMP and how does ping use it?

Ans: ICMP (Internet Control Message Protocol) carries error and control messages of IP. Ping sends ICMP echo request (type 8) and receives echo reply (type 0) to test reachability and measure round-trip time.

Q14. What do TTL values 128, 64 and 56 in this experiment indicate?

Ans: TTL is the hop limit of an IP packet. Windows starts at 128 and Linux-based routers at 64, and each router decreases it by 1. The reply of the gateway has TTL 64 (no hop), while the reply from 1.1.1.1 has TTL 56, so the packet crossed 8 routers.

Q15. Explain the TCP three-way handshake.

Ans: (1) The client sends SYN with its initial sequence number; (2) the server replies SYN-ACK with its own sequence number and an acknowledgment of the client's number + 1; (3) the client sends ACK. The connection is then established.

Q16. What is the use of the TCP flags SYN, ACK, PSH, FIN and RST?

Ans: SYN starts a connection, ACK acknowledges data, PSH asks to deliver data immediately, FIN closes a connection normally and RST abruptly resets it.

Q17. What is the difference between TCP and UDP?

Ans: TCP is connection-oriented and reliable (handshake, sequence numbers, acknowledgments, retransmission, flow control) with a 20–60 byte header. UDP is connectionless, unreliable and fast with an 8-byte header; it is used for DNS queries, streaming and calls.

Q18. Why was the HTTP packet readable but not the bigbasket packets?

Ans: httpforever.com uses plain HTTP on port 80, so the request and response are sent in clear text. bigbasket.com uses HTTPS (TLS on port 443), which encrypts the data, so Wireshark shows only TLS "Application Data".

Q19. What does the HTTP status 304 Not Modified mean?

Ans: The requested file has not changed since the browser last downloaded it, so the server sends no body and the browser uses its cached copy.

Q20. What is a port number? Name a few well-known ports.

Ans: A port number (16 bits) identifies the application on a host. HTTP – 80, HTTPS – 443, DNS – 53, FTP – 21, SSH – 22, SMTP – 25. Here 3478 (STUN/TURN) was seen in UDP.

Q21. What is the maximum segment size (MSS)?

Ans: MSS is the largest amount of data in a single TCP segment; it is announced in the SYN packet as a TCP option (1460 bytes here, for an MTU of 1500).

Q22. What is relative sequence number in Wireshark?

Ans: Wireshark shows sequence numbers relative to the start of the connection (starting from 0 at the SYN) to make them easier to read; the real 32-bit values are shown as "raw".

Q23. What is the use of the Transaction ID in DNS?

Ans: It is a 16-bit number copied from the query into the response, so the client can match each response with the right query.

Q24. Is the ARP request always a broadcast?

Ans: Normally yes (destination ff:ff:ff:ff:ff:ff). In this capture the router sent the request as unicast to the laptop's MAC to verify an existing ARP entry.